

AE 598 HW2: Model-Free Tabular Methods

William Schafer

University of Illinois Urbana-Champaign, Champaign, IL 61820

I. Introduction

In reinforcement learning, model-free tabular methods define a set of methods that learn an optimal policy π^* from direct interaction with an environment defined by a finite state space S and action space A without any prior knowledge of its dynamics $P(s'|s, a)$. The tabular setting of the environment allows estimations of the policy π to be explicit such that every state has a defined action. The goal of these methods is to approximate an optimal policy π^* that maximizes the expected discounted return at every state within a defined Markov Decision Process (MDP).

Since the environment dynamics $P(s'|s, a)$ are unknown, model-free tabular methods collect experience through episodes, defined as the set of all states visited and actions taken from the start state to a terminal state. During or after each episode, they estimate the discounted return following each visited state-action pair (s, a) , yielding the action-value function estimate $Q(s, a)$ for that episode. A new policy π' is then greedily obtained by choosing the highest-value action at every state within $Q(s, a)$, shown in Eq. 1. The cycle then repeats utilizing π' , improving after each episode. The optimality of the generated π is thereby dependent on the number of episodes employed during the iteration process, where a larger number of episodes typically leads to a better estimation of the optimal policy π^* .

This report focuses on three of these methods: Monte Carlo (MC) control, SARSA (on-policy TD(0)), and Q-Learning (off-policy TD(0)), applied to the Frozen Lake MDP with discount factor $\gamma = 0.95$, using an ϵ -greedy policy improvement step to maintain exploration throughout training.

$$\pi'(s) = \arg \max_a Q(s, a). \quad (1)$$

All models, equations, and algorithms in this report are derived from Sutton & Barto [1]. The Frozen Lake MDP environment is simulated via OpenAI Gym's Gymnasium [2].

A. The Frozen Lake MDP

The Frozen Lake MDP defines a 4×4 discrete state space where the top-left cell at $[0, 0]$ is the start S , the bottom-right cell at $[3, 3]$ is the goal G , cells $[1, 1]$, $[1, 3]$, $[2, 3]$, and $[3, 0]$ are holes H , and every other cell is frozen F . The states are indexed in row-major order, so $s \in \{0, \dots, 15\}$ with $S = 0$ and $G = 15$. The action space is $A = \{0, 1, 2, 3\}$ with 0: left, 1: down, 2: right, 3: up. The dynamics $P(s' | s, a)$ move the agent one cell in the commanded direction unless that move is off-grid, in which case the agent remains in place. When the ice is slippery (`is_slippery=True`), the agent executes the intended direction with probability $1/3$ and each perpendicular direction with probability $1/3$. An episode terminates upon entering a hole or the goal. The reward is defined as

$$R(s, a, s') = \begin{cases} 1, & s' = G, \\ 0, & \text{otherwise} \end{cases}$$

Thus, the MDP is defined as

$$\text{MDP} = (S, \mathcal{A}, P, R, \gamma) = \left(\begin{bmatrix} 0 \\ 1 \\ 2 \\ \vdots \\ 15 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix}, P(s' | s, a), R(s, a, s'), 0.95 \right), \quad \text{map} = \begin{bmatrix} S & F & F & F \\ F & H & F & H \\ F & F & F & H \\ H & F & F & G \end{bmatrix}.$$

In the model-free methods considered in this report, $P(s' | s, a)$ is unknown and learning proceeds purely from interaction within the environment.

B. Monte Carlo (MC) Control

Monte Carlo (MC) control is a model-free method that estimates the action-value function $Q(s, a)$ by averaging the first-visit return observed after (s, a) in sampled episodes, and then greedily extracting a new policy π' from $Q(s, a)$ following Eq. 1. Returns are computed only for the first occurrence of a state-action pair within an episode. For an episode $(S_0, A_0, R_1, \dots, S_T)$, the first-visit return from time t is defined in Eq. 2.

$$G_t \doteq \sum_{k=0}^{T-t-1} \gamma^k R_{t+1+k}. \quad (2)$$

After each episode, every first-visited (S_t, A_t) appends G_t to a per-pair list $\text{Returns}(S_t, A_t)$. The action-value estimate is the average over that list, defined in Eq. 3.

$$Q(s, a) = \text{average}(\text{Returns}(s, a)). \quad (3)$$

A new policy π' is then extracted from $Q(s, a)$ following Eq. 1. The new policy is used for the next episode, and the evaluate-improve cycle repeats over many episodes, yielding π^* after some large number of episodes.

C. SARSA (On-Policy TD(0))

Unlike MC control which updates $Q(s, a)$ after each episode (offline), SARSA (state-action-reward-state-action) is a model-free, on-policy temporal-difference method that actively updates the action-value function $Q(s, a)$ after each step during an episode (online) using the one-step target produced by the current policy. For a transition $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, the target is defined in Eq. 2 and the corresponding $Q(s, a)$ update is given in Eq. 3.

$$\text{target} \doteq R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) \quad (2)$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(\text{target} - Q(S_t, A_t)) \quad (3)$$

Here, α is the learning rate: the update can be viewed as taking the old $Q(s, a)$ and adding a step in the direction of the reward gradient. The learning rate α scales how strongly new information adjusts Q . Larger α adapts quickly but is noisier and more sensitive, while smaller α changes Q more cautiously. Actions A_t and A_{t+1} are chosen on-policy, meaning that they follow the defined π or, in this case, an ϵ -greedy extraction of the highest action-value at S_t . After many episodes, and many updates to $Q(s, a)$, a policy π^* is extracted greedily from $Q(s, a)$ using Eq. 1.

D. Q-Learning (Off-Policy TD(0))

Q-Learning is the off-policy implementation of SARSA, actively updating the action-value function $Q(s, a)$ after each step online using a greedy target that does not depend on the action actually taken next. For a transition $(S_t, A_t, R_{t+1}, S_{t+1})$, the target is defined in Eq. 4 and the corresponding $Q(s, a)$ update is given in Eq. 5.

$$\text{target} \doteq R_{t+1} + \gamma \max_a Q(S_{t+1}, a) \quad (4)$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(\text{target} - Q(S_t, A_t)) \quad (5)$$

During learning, the agent executes ϵ -greedy actions from the current $Q(s, a)$. After executing A_t and observing R_{t+1} and S_{t+1} , Q-Learning updates the value of the action actually taken, $Q(S_t, A_t)$, using the purely greedy action at that state, $\max_a Q(S_{t+1}, a)$, rather than the actual next action defined at the next state, $Q(S_{t+1}, A_{t+1})$ which typically has some ϵ chance to explore (random action rather than greedy action). This difference between the behavior and the target makes Q-Learning off-policy. Typically, Q-Learning converges to the optimal action-value function $Q^*(s, a)$, after which a policy π^* is extracted greedily from Q using Eq. 1.

II. Methodology

A. Monte Carlo (MC) Control

Before MC control could be implemented, a function for computing the ϵ -greedy policy π and a function for computing the first-visit returns G had to be made. Following Eq. 1 with the added ϵ chance for a random action to be chosen, the function for computing the ϵ -greedy policy π was implemented in pseudo-code as Alg. 1.

Algorithm 1 Extract Epsilon-Greedy Policy from Action-Value Function

```
1 def getPiFromQ(Q, epsilon):
2     nStates, nActions = Q.shape
3     pi = zeros(nStates)
4     for s in range(nStates):
5         greedyAction = max(Q[s])
6         pi[s] = greedyAction
7         if random([0, 1]) < epsilon: pi[s] = random([0, nActions])
8     return pi
```

Following Eq. 2 for computing the first-visit returns G , the equation was implemented in pseudo-code as Alg. 2.

Algorithm 2 Compute First-Visit Returns for an Episode

```
1 def getFirstVisitReturns(gamma, R, S, A):
2     nStates, nActions = len(S), len(A)
3     G = zeros(nStates, nActions)
4     visited = []
5     Gt = 0
6     for step in reversed(range(nActions)):
7         Gt = R[step] + gamma*Gt
8         s, a = S[step], A[step]
9         if (s,a) not in visited:
10             visited.append((S,a))
11             G[s, a] = Gt
12     return G
```

which recursively computes the discounted returns of every first-visit state-action pair from a given episode. Finally, following Eq. 3 and adding Algs. 1 and 2 into the loop, MC control was implemented in pseudo-code as Alg. 3.

Algorithm 3 Monte Carlo (MC) Control

```
1 def MCcontrol(nEpisodes, gamma, epsilon, is_slippery):
2     Q = zeros(16, 4) # 16 states, 4 actions
3     Returns = []
4     for _ in range(nEpisodes):
5         pi = getPiFromQ(Q, epsilon)
6         S, A, R, _ = simFrozenLakeMDP(pi, is_slippery)
7         G = getFirstVisitReturns(gamma, R, S, A)
8         Returns.append(G)
9         Q = average>Returns)
10     pi = max(Q)
11     return pi
```

B. SARSA (On-Policy TD(0))

Due to SARSA's online nature, it is inefficient to generate a new policy π at every step. So, instead, the ϵ -greedy action is extracted from the current action-value function $Q(s, a)$ at every step. This was implemented into pseudo-code as Alg.4.

Algorithm 4 Extract Epsilon-Greedy Action from Action-Value Function at a State

```
1 def getActionFromQ(Q, s, epsilon):
2     nActions = Q.shape[1]
3     greedyAction = max(Q[s])
4     if random([0, 1]) < epsilon: greedyAction = random([0, nActions])
5     return greedyAction
```

The action-value function $Q(s, a)$ update as defined in Eq. 3 was then implemented as pseudo-code in Alg. 5.

Algorithm 5 SARSA Q Update

```
1 def sarsaQupdate(Q, s0, a0, s1, a1, r, alpha, gamma, terminal=False):
2     target = r + gamma * Q[s1, a1]
3     if terminal: target = r
4     Q[s0, a0] += alpha * (target - Q[s0, a0])
5     return Q
```

Finally, the SARSA method was implemented in pseudo-code as Alg. 6.

Algorithm 6 SARSA

```
1 def SARSA(nEpisodes, gamma, epsilon, alpha, is_slippery, max_steps):
2     Q = zeros(16, 4) # 16 states, 4 actions
3     for _ in range(nEpisodes):
4         environment = make('FrozenLake', is_slippery)
5         s, _ = environment.reset()
6         a = getActionFromQ(Q, s, epsilon)
7         for _ in range(max_steps):
8             s_next, r, terminated, _ = environment.step(a)
9             a_next = getActionFromQ(Q, s_next, epsilon)
10            Q = sarsaQupdate(Q, s, a, s_next, a_next, r, alpha, gamma, terminated)
11            if terminate: break
12            s, a = s_next, a_next
13     pi = max(Q)
14     return pi
```

C. Q-Learning (Off-Policy TD(0))

The Q-Learning algorithm is implemented using the exact same code as the SARSA algorithm, except with a tweak to the $Q(s, a)$ update in Alg. 5. Q-Learning uses the purely greedy action at S_{t+1} to update $Q(s, a)$ instead of the followed policy (an ϵ -greedy policy in this report). Thus, using Alg. 4, the Q-Learning update is defined as pseudo-code in Alg. 7.

Algorithm 7 Q-Learning Q Update

```
1 def QLearningQupdate(Q, s0, a0, s1, r, alpha, gamma, terminal=False):
2     a1_greedy = getActionFromQ(Q, s1, epsilon=0)
3     target = r + gamma * Q[s1, a1_greedy]
4     if terminal: target = r
5     Q[s0, a0] += alpha * (target - Q[s0, a0])
6     return Q
```

The Q-Learning algorithm is then defined exactly as Alg 6, only replacing line 10 with Alg. 7.

III. Results

A. Learning Curves

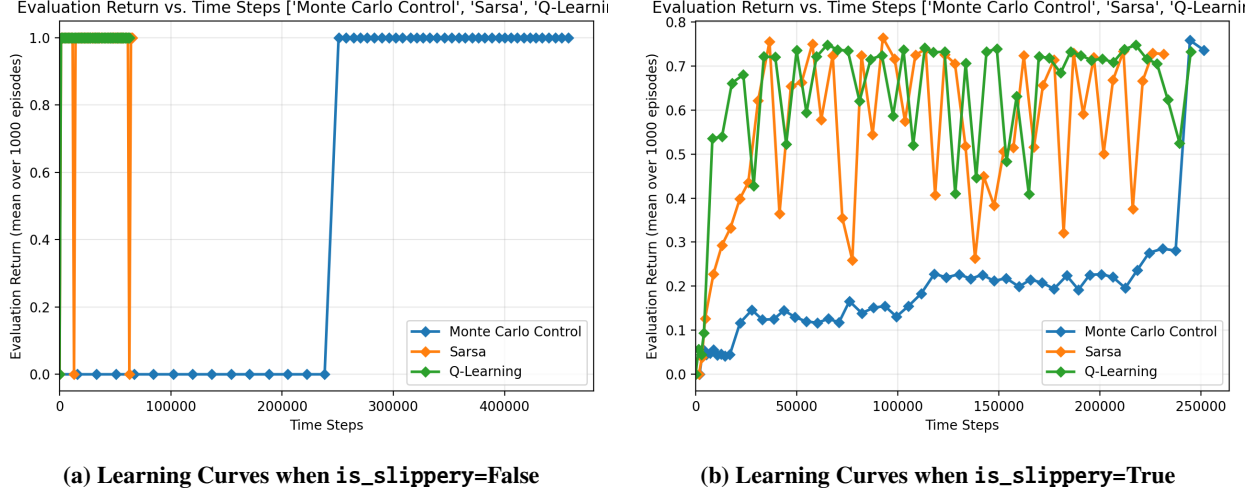


Fig. 1 The learning curves of MC Control, SARSA, and Q-Learning evaluated over 1,000 episodes.

With fixed hyperparameters ($\epsilon = 0.1$, $\alpha = 0.1$, $\gamma = 0.95$), the learning curves in Fig. 1 show the expected behavior. In the non-slippy setting, SARSA and Q-Learning reach a return of 1.0 almost immediately, whereas Monte Carlo (MC) needs roughly 250,000 time steps to catch up, which is consistent with MC's episodic (offline) updates versus the step-wise temporal difference (TD) updates used by SARSA and Q-Learning. After convergence, SARSA exhibits brief dips around 1,000 and 6,000 steps, which stem from continual ϵ -greedy exploration combined with a relatively large step size ($\alpha = 0.1$) that makes on-policy updates more sensitive to noisy values. In the slippy setting, where failure can occur despite optimal policies, all methods approach the expected return of roughly 0.7. Q-Learning stabilizes first (25,000 steps), SARSA follows (35,000 steps), and MC improves much more slowly, nearing the steady-state only near the end of its evaluation (240,000 steps). After stabilizing, both TD methods show occasional drops, with SARSA's being deeper because it learns from on-policy, which is exploratory, while Q-Learning uses a greedy off-policy target that is less perturbed by exploration. From testing, lowering the learning rate α reduced the frequency and depth of these dips since the algorithm relied less on any single noisy value, but also required more interaction to converge to the same policy. Overall, step-wise TD methods learn significantly faster than MC, and once converged, MC is the most stable.

B. Final Policies

For the policy maps shown in the top row of Fig 2, the non-slippy case is straightforward: all three algorithms produced the same optimal policy π^* . With no chance of unintended motion, every intended move leads to the intended next state, so π^* simply marches toward the goal while avoiding holes, and there is no ambiguity. In the slippy case shown in the bottom row of Fig. 2, each method still achieved an optimal policy, but the resulting maps differed because multiple actions are equally safe, and thus equally optimal, in a multitude of the states. The only inherently risky location is state 6 (the one-cell corridor with holes on both sides). There, any action aimed along the corridor yields a $2/3$ chance of falling into a hole, so the optimal action is to move directly towards a hole, yielding a $2/3$ chance of safety with only a $1/3$ chance of falling into a hole. Everywhere else, a 100% safe move exists even with slip. Consequently, while the value functions agree, policies can choose different but equally optimal safe moves. This is shown at state 3 across the policy maps in the bottom row of Fig. 2, where one method goes right, another down, and another left. These differences reflect the tie-breaking between equally optimal actions (randomly choosing an action if equal in value to another at a given state) and exploration due to $\epsilon = 0.1$ during learning rather than sub-optimality.

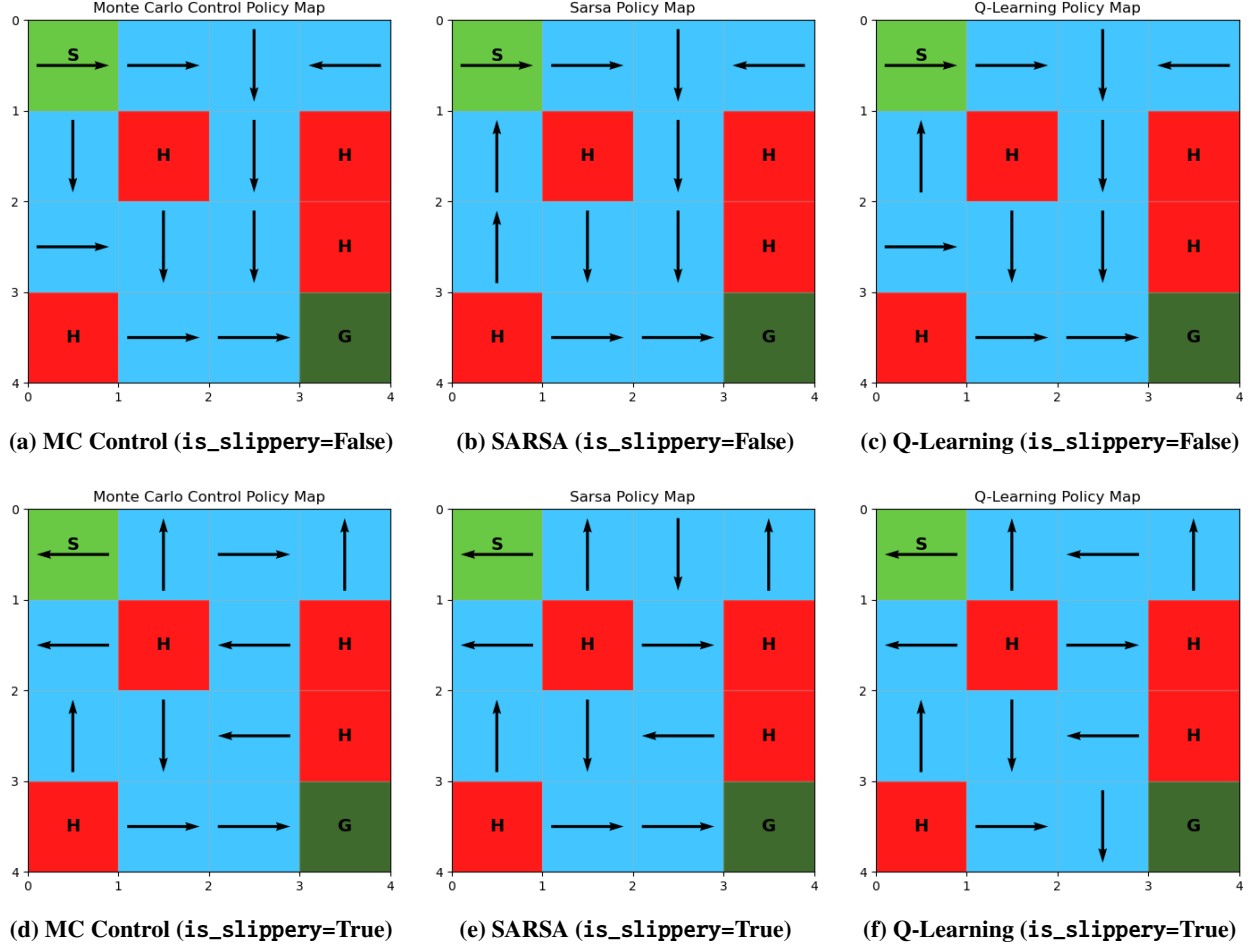


Fig. 2 Final policies for MC Control, SARSA, and Q-Learning in the Frozen Lake MDP.

IV. Conclusion

This report implemented Monte Carlo control, SARSA, and Q-Learning on the 4×4 Frozen Lake. In the non-slippery setting, all methods learned the same optimal policy π^* . SARSA and Q-Learning converged almost immediately, while MC required more steps but was most stable once converged. In the slippery setting, all approaches rose toward the expected return of roughly 0.7. Q-Learning stabilized first, SARSA shortly after, and MC much later. The occasional post-convergence dips in the TD curves stem from ϵ -greedy exploration combined with a relatively large α . Lowering α reduced these dips but required more interaction to reach the same policy. Policy maps confirmed identical optimal actions without slip and multiple, equally optimal choices with slip, showing that step-wise temporal difference methods learn faster while MC offers greater stability after convergence.

References

- [1] Sutton, R. S., and Barto, A. G., *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, Cambridge, MA, 2018.
- [2] Gymnasium, “Frozen Lake,” https://gymnasium.farama.org/environments/toy_text/frozen_lake, 2025. Accessed: 2025-10-01.